

Architecture Upgrade Proposal

Cloud-Based Research Paper Summarization Platform

Pranjal Mishra

April 15, 2026

Summary

After reviewing the Week 2 architecture, I'd like to propose a set of improvements. They fall into two groups: **required changes** that fix real issues in the current design, and **nice-to-haves** that make the system stronger if we have time. Each item below explains what the change is, the problem it solves, and roughly how much work it adds.

Required Changes

These are the seven changes I think we should lock in before starting implementation in Week 3.

1. Move authentication to the API Gateway layer

Current design. The diagram shows our Lambda functions talking to Cognito to check tokens.

Change. Move JWT validation to API Gateway using a Cognito Authorizer, so tokens are checked *before* Lambda is ever invoked.

Why this matters. Any unauthenticated request should never reach our business logic — it's a waste of compute and a subtle security concern. This is also the standard AWS pattern; every AWS tutorial and textbook shows this direction. Keeping the current flow would raise an eyebrow from anyone who knows AWS.

Effort. Five minutes. It's a single configuration change in the API Gateway.

2. Put CloudFront in front of the S3 frontend

Current design. S3 serves the frontend directly.

Change. Add Amazon CloudFront as the public entrypoint; make the S3 bucket private so only CloudFront can reach it.

Why this matters. Three things we lose by serving directly from S3: HTTPS on our own domain (S3 can't do this on a custom domain without CloudFront), edge caching (every user

worldwide hits one region), and DDoS protection. CloudFront gives us all three for free, and it's a single resource in our IaC. Any production web app on AWS does this.

Effort. One hour to set up, basically no ongoing work.

3. Use Step Functions instead of a single Lambda for the summarization pipeline

Current design. One Lambda handles the whole summarization.

Change. Replace it with an AWS Step Functions state machine with five steps: FetchPDF → ExtractText → Chunk → MapSummarize (parallel) → ReduceSummary. Each step is a small focused Lambda.

Why this matters. This is the biggest reliability win in the whole design.

- A single Lambda has a hard 15-minute timeout. A 40-page paper going through map-reduce summarization with Claude can easily exceed this. Right now we'd just fail and the user would see an error.
- Step Functions retries each step individually if it fails. If Bedrock hits a rate limit during chunk 4 of 12, we retry that one chunk — we don't redo the whole paper.
- Step Functions gives us a visual execution history. When something fails, we can literally see which step broke and why. That makes debugging fast.
- If we ever want to add a step (like caching or enrichment), we just add another state. The code of each step Lambda stays focused and simple.

Effort. Medium. The Lambdas themselves are what we'd write anyway; Step Functions is just the glue that connects them.

4. Add a Dead Letter Queue to SQS

Current design. One SQS queue, no DLQ.

Change. Configure the queue with a redrive policy: after three failed deliveries, the message moves to a separate DLQ. Add a CloudWatch alarm on DLQ depth > 0 so we know if anything lands there.

Why this matters. Without a DLQ, if summarization keeps failing on a specific paper (say it's an encrypted PDF we can't read), SQS will retry forever, spend money, and eventually drop the message. The user will see their job stuck in "pending" with no explanation. A DLQ catches these and lets us investigate.

Effort. Ten minutes. One extra resource in CDK.

5. Use Amazon Bedrock instead of the Anthropic API directly

Current design. Direct calls to the Anthropic API.

Change. Call Claude Sonnet through Amazon Bedrock.

Why this matters. Same model quality, but three practical wins:

- **No API key management.** With Bedrock, Lambda calls the model using its IAM role. There's no secret to store, rotate, or accidentally leak in a commit.
- **It's all in one AWS bill and one CloudWatch dashboard.** Direct Anthropic means a second account, second billing relationship, and separate observability.
- **It fits the AWS-native story of our project.** The course is on cloud computing; using one cloud provider end-to-end is cleaner.

The only thing we need to check: Bedrock has to be enabled in our AWS region (us-east-1). If our educational account doesn't have Bedrock access, we fall back to Anthropic direct — but let's verify before deciding.

Effort. Few lines of code change in one Lambda, assuming Bedrock access is available.

6. Document our DynamoDB access patterns up front

Current design. We chose DynamoDB but didn't specify how we'll structure it.

Change. Commit to a single-table design now, with defined keys:

- Partition key: USER#{userId}
- Sort keys: PROFILE#, JOB#{jobId}, SUMMARY#{timestamp}
- A Global Secondary Index keyed on PAPER#{doi} so we can check if a paper has been summarized before (by anyone) and reuse it — saving Bedrock costs.

Why this matters. DynamoDB is schemaless, which means if we don't think about access patterns up front we'll end up doing full table scans (slow and expensive) or migrating later. Ten minutes of design now saves hours of pain later. Also, any code reviewer or instructor will ask us "what are your access patterns?" — we should have the answer ready.

Effort. Thirty minutes of design. No code impact; it's just the schema we'll code against.

7. Adopt AWS CDK for Infrastructure-as-Code

Current design. Not specified.

Change. Use AWS CDK in TypeScript for all infrastructure. Every piece of AWS config lives in version-controlled code, not in someone's browser.

Why this matters. Four developers on four laptops each clicking around the AWS console is chaos:

- We can't review each other's infrastructure changes.
- If someone accidentally deletes something in the console, there's no way to recover.
- When we want to demo or test on a clean environment, we'd have to click through everything again.
- If we move to a grader's AWS account, we can't easily redeploy.

With CDK, our entire infrastructure is `git clone && cdk deploy`. We can give a grader the repo and they can spin up their own copy.

Effort. About one day to scaffold. After that, every infrastructure change is a normal pull request and saves time every week.

Nice-to-Haves

If we have capacity in the later weeks, these are worth considering. None are urgent.

8. Secrets Manager for any third-party keys

If we ever use any external API key (like Semantic Scholar if we add it, or a fallback LLM), it belongs in AWS Secrets Manager with rotation — not in environment variables. Cheap insurance against accidental leaks.

9. Observability baseline

Structured JSON logs from every Lambda (so we can search them later), alarms on Lambda error rates and DLQ depth, and one shared CloudWatch dashboard. About half a day of setup.

10. S3 lifecycle policy on the PDF bucket

Auto-delete raw PDFs after 90 days. Keeps our bill flat and avoids any long-term copyright question about hoarding papers.

11. Per-user quota

A simple counter stored on each user's Cognito profile, decremented when they submit a summary. Prevents one user (or a bug) from draining our Bedrock credits in a single afternoon.

12. AWS Budgets alarm

One email alert if we cross \$X/month. Takes five minutes. There's no reason not to have this.

13. Summary idempotency / deduplication

Hash the PDF + user; check DynamoDB before submitting. If we already summarized that paper recently, return the cached result instead of calling Bedrock again. Saves tokens, makes repeat requests feel instant.

Stretch Goals for the Final Demo

Purely optional, but these are the things that would make our demo stand out:

- **AWS WAF** on CloudFront to block bots and abusive clients
- **WebSocket push notifications** so the frontend gets “summary ready!” in real time instead of polling
- **“Chat with this paper”** feature using embeddings stored in OpenSearch Serverless or pgvector
- **CI/CD with GitHub Actions** deploying a test stack per pull request
- **AWS X-Ray** traces for end-to-end latency debugging

Summary Table

Tier	Count	Combined Effort
Required (items 1–7)	7	Roughly one week total
Nice-to-haves (items 8–13)	6	Incremental, 2–3 days
Stretch goals	5	Only if time permits

What I’d Like From the Team

1. Quick yes/no on the seven required items. If anyone sees a problem with one, let’s discuss before I update the architecture doc.
2. Which (if any) of items 8–13 do we commit to?
3. Any stretch goals anyone wants to champion for the demo?

Once we’re aligned, I’ll update the architecture diagram and the Week 2 report.